

LEADRS DRE - Multi-State Hierarchy Plan

Version: 1.0 Date: 2026-01-20 Status: Ready for Implementation

Executive Summary

LEADRS DRE is expanding from **Texas-only** to a **multi-state platform** with complete data isolation between states.

Current	Future
Only Texas (TMPA)	Multiple states (Texas, Florida, California, etc.)
All data in one pool	Each state's data completely isolated
TMPA is root org	TMPA becomes Platform Root, states below

Key Points:

- Zero impact for Texas users - Same login, screens, workflow
- Automatic data isolation - Enforced at database level
- Easy to add new states - Simple 4-step process

Part 1: Approach

Option 2: Add New Platform Root at Top (Chosen)

BEFORE :	AFTER :
TMPA (OrgId=1) ├ Dallas PD (ParentId=1) ├ Houston PD (ParentId=1) └ Austin PD (ParentId=1)	TMPA (OrgId=NEW) ← NEW Platform Root ├ Texas State (OrgId=1, was TMPA) │ ├── Dallas PD (ParentId=1) ← UNCHANGED │ └ Houston PD (ParentId=1) ← UNCHANGED

Why Option 2?

Benefit	Description
Minimal changes	Only 1 FK update vs 100+

Low risk	Easy rollback if needed
No code breaks	Existing OrgId=1 references still work
SP unchanged	Organization_Children returns same results

Part 2: Architecture

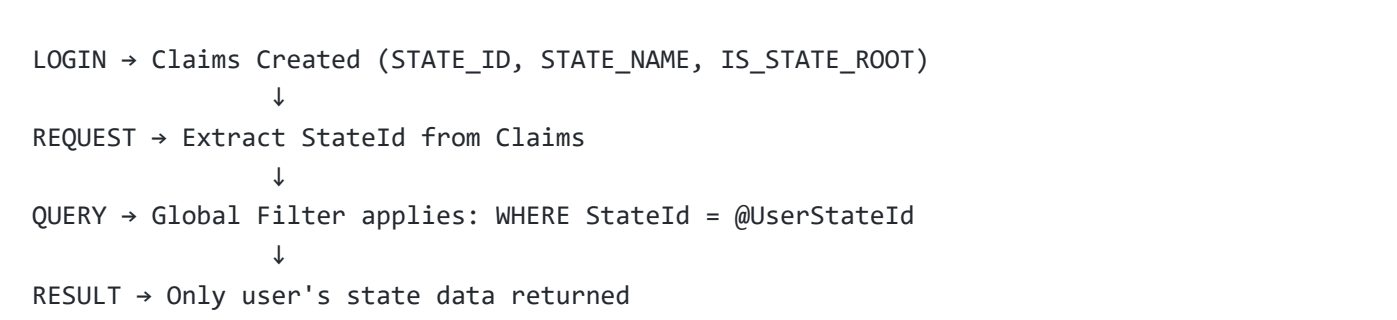
Hierarchy Levels

Level	Name	Example	StateId	Access
0	Platform Root	TMPA (new)	NULL	Read-only all states
1	State Root	Texas State, FLPA	Self (OrgId)	Full control within state
2	Organization	Dallas PD, Miami PD	Parent State's ID	Isolated within state

Access Control

User Type	Own State	Other States	Write
Officer	✓ Read	X	Own Org
State Admin	✓ Read	X	All in State
Platform Admin	✓ Read	✓ Read-only	X None

Data Flow



Part 3: Key Components

Component	Purpose
StateId column	Added to Organization, Users, DRECase, and all state-scoped tables
IsStateRoot flag	Marks state-level root organizations
IStateContext service	Reads StateId from user claims
IStateResource interface	Marker for state-scoped entities
Global Query Filters	Auto-applies WHERE StateId=X to all queries
StateAuthorizationHandler	Enforces state isolation

New Claims

STATE_ID → User's state (e.g., 1 for Texas)
 STATE_NAME → "Texas State"
 IS_STATE_ROOT → true if state admin

Code Patterns

```
// Get StateId
int stateId = User.GetStateId();
int stateId = _stateContext.StateId;

// Query (auto-filtered by EF Core)
var cases = await _context.DRECases.ToListAsync();
// → Generates: SELECT * FROM DRECase WHERE StateId = @UserStateId

// Bypass for Platform Root (read-only)
var all = await _context.DRECases.IgnoreQueryFilters().AsNoTracking().ToListAsync();
```

Stored Procedures

Add @StateId to **ALL** stored procedures for safety and consistency.

```
-- Example: Organization_Children
ALTER PROCEDURE [dbo].[Organization_Children]
    @ParentId INT,
    @StateId INT -- Added for safety & consistency
```

```
AS
BEGIN
    SELECT * FROM Organization
    WHERE ParentOrganizationId_FK = @ParentId
    AND StateId = @StateId
END
```

Note: ParentId manages hierarchy by itself, but passing StateId adds extra safety and keeps all SPs consistent.

Part 4: 3-Phase Rollout

Overview

Phase	What	User Impact
1	Database + Entity changes (no filters)	NONE
2	Claims + Query Filters + Authorization	NONE for Texas
3	COMS Integration + Multi-State	New states can join

Phase 1: Foundation (Database + Data Only)

Goal: Add StateId columns, migrate data. NO behavior changes.

Database Changes

Step	Task
1	Backup database
2	Add StateId (INT NULL), IsStateRoot (BIT) to Organization table
3	Create NEW Platform Root org ("TMPA", StateId=NULL)
4	Rename existing TMPA to "Texas State", set ParentId to Platform Root, StateId=1
5	Set StateId=1 for all existing organizations
6	Add StateId toAspNetUsers, set all to 1

7	Add Stateld to DRECase and all state-scoped tables
8	Create indexes on Stateld columns
9	Verify migration

Entity Changes

File	Change
Organization.cs	Add Stateld, IsStateRoot properties
ApplicationUser.cs	Add Stateld property
DRECase.cs	Add Stateld property
All state-scoped entities	Add Stateld property

User Impact: NONE - Application works exactly as before.

Phase 2: Enable State Isolation

Goal: Enable automatic state filtering. Texas users see no change.

Changes

Step	Task
1	Create IStateResource interface
2	Create IStateContext / StateContext service
3	Add ClaimsEnum values (STATE_ID, STATE_NAME, IS_STATE_ROOT)
4	Add claims extension methods (GetStateld, IsStateRoot)
5	Update token generation with state claims
6	Add global query filters in DbContext
7	Override SaveChangesAsync to auto-set Stateld
8	Create StateAuthorizationHandler
9	Update stored procedures with @Stateld parameter

User Impact: NONE for Texas users - data already belongs to Texas.

Phase 3: Multi-State Support

Goal: Enable adding new states.

COMS Changes (First)

Step	Task
1	Add StatelId to COMS Organization table
2	Add StatelId to COMS User table
3	Migrate existing data (StatelId= 1 for Texas)
4	Add IStateContext service (default to Texas)
5	Add global query filters
6	Update API responses to include StatelId
7	Deploy COMS

DRE Changes (Second)

Step	Task
8	Update sync services to receive StatelId from COMS
9	Add StatelId to token claims
10	Enable new state creation workflow
11	Deploy DRE

User Impact: NONE for existing users. New states can be added.

Part 5: COMS Integration

Strategy: Default to Texas

When an app calls COMS API **without StatelId**, it defaults to Texas (StatelId= 1).

Scenario	Result
----------	--------

DRE passes Stateld	Gets filtered data for that state
Other apps (no Stateld)	Gets Texas data (safe default)
Platform Root (flag=true)	Gets all states data (read-only)

Why Default to Texas?

- COMS currently only serves Texas data
- All existing apps only need Texas data
- More secure - no accidental cross-state data exposure
- No breaking changes

Part 6: Files to Modify/Create

Files to Create

File	Purpose
IStateContext.cs	State context interface
StateContext.cs	State context implementation
IStateResource.cs	Entity marker interface
StateAuthorizationHandler.cs	Authorization handler

Files to Modify

File	Change
Organization.cs	Add Stateld, IsStateRoot
ApplicationUser.cs	Add Stateld
DRECase.cs	Add Stateld, implement IStateResource
ClaimsEnum.cs	Add STATE_ID, STATE_NAME, IS_STATE_ROOT
ClaimsPrincipalExtensions.cs	Add GetStateld(), IsStateRoot() methods
ApplicationDbContext.cs	Add global query filters
DependencyConfig.cs	Register IStateContext

Token generation code	Add state claims
All stored procedures	Add @Stateld parameter

Part 7: Testing Checklist

Phase 1 Tests

- ☐ Database backup taken
- ☐ Platform Root org created
- ☐ Existing TMPA renamed to "Texas State"
- ☐ All Texas orgs have Stateld = 1
- ☐ Sub-org ParentId references UNCHANGED
- ☐ Application starts without errors
- ☐ Existing functionality works as before

Phase 2 Tests

- ☐ Login sets STATE_ID claim correctly
- ☐ Queries automatically filter by Stateld
- ☐ New records get Stateld from claims
- ☐ Texas users see only Texas data
- ☐ Organization_Children SP works with Stateld parameter

Phase 3 Tests

- ☐ COMS → DRE sync includes Stateld
- ☐ New state can be created
- ☐ New state users isolated from Texas
- ☐ Texas users isolated from new state
- ☐ Platform Root sees all states (read-only)

Part 8: Rollback Procedures

Phase 1 Rollback

```
BEGIN TRANSACTION;

-- Get Platform Root OrgId
DECLARE @PlatformRootId INT;
SELECT @PlatformRootId = OrganizationId
FROM Organization WHERE StateId IS NULL AND IsStateRoot = 1;

-- Rename Texas State back to TMPA
UPDATE Organization
SET OrganizationName = 'TMPA', ParentOrganizationID_FK = NULL
WHERE OrganizationId = 1;

-- Delete Platform Root
DELETE FROM Organization WHERE OrganizationId = @PlatformRootId;

-- Drop StateId columns
ALTER TABLE DRECase DROP COLUMN StateId;
ALTER TABLE AspNetUsers DROP COLUMN StateId;
ALTER TABLE Organization DROP COLUMN StateId, IsStateRoot;

COMMIT TRANSACTION;
```

Phase 2 Rollback

1. Remove global query filters from DbContext
2. Remove state claims from token generation
3. Remove IStateContext registration
4. Revert stored procedures
5. Deploy

Part 9: FAQ

For Users

Q: Will my login change?

No. Same username and password.

Q: Will the screens look different?

No. Everything looks and works exactly the same.

Q: Will I see other state's data?

No. You will only see your state's data.

Q: Do I need training?

No. There is nothing new to learn.

For Administrators

Q: How do we add a new state?

Platform admin creates the state in COMS, then state admin adds departments and users.

Q: Can one person work in multiple states?

Each user belongs to one state. Separate accounts needed for multiple states.

Q: Who can see all states' data?

Only Platform Admin, and only for reports (read-only, cannot edit).

For Developers

Q: Do I need to add StateId filters to my queries?

No. Global query filters handle this automatically via EF Core.

Q: What about stored procedures?

Add `@StateId` parameter to ALL stored procedures and pass `_stateContext.StateId` from code.

Q: Does Organization_Children SP need updates?

Yes, add `@StateId` parameter. While `ParentId` manages hierarchy by itself, passing `StateId` is recommended for added safety and consistency across all stored procedures.

Part 10: Quick Reference

HIERARCHY:

```
TPMA (Platform Root, StateId=NULL)
└─ Texas State (OrgId=1, StateId=1) ← was "TPMA"
    └─ Dallas PD (ParentId=1) ← UNCHANGED
```

KEY RULES:

- Platform Root: StateId = NULL
- State Root: StateId = OrganizationId (self)
- Child Org: StateId = State Root's OrgId

GET STATE INFO:

```
int stateId = User.GetStateId();
int stateId = _stateContext.StateId;
```

QUERY FILTER (Auto):

```
var cases = await _context.DRECases.ToListAsync();
// → SELECT * FROM DRECase WHERE StateId = @UserStateId
```

STORED PROCEDURE:

```
var stateId = _stateContext.StateId;
await _context.Database.ExecuteSqlRawAsync(
    "EXEC [dbo].[SP] @Param={0}, @StateId={1}", param, stateId);
```

PHASES:

```
Phase 1: Database + Entity changes (no filters)
Phase 2: Claims + Query Filters + Authorization
Phase 3: COMS Integration + Multi-State
```

Document Version: 1.0 Last Updated: 2026-01-20